Library Management System

🎯 Project Overview

You will design and implement a complete database system for **City Central Library**, a medium-sized public library that wants to modernize their book lending and membership management. This project will test ALL the concepts you've learned over the past 7 days.

📋 Problem Statement

City Central Library serves 5,000+ members and manages 20,000+ books across multiple genres. They need a robust database system to:

- Track books, authors, and publishers
- Manage member registrations and memberships (with expiry dates)
- Handle book borrowing and returns
- Track fines for overdue books
- Generate reports on popular books, member activity, and revenue
- Manage multiple library branches

🔧 Project Requirements

Task 1: ERD Design

Design a complete Entity-Relationship Diagram that includes:

Minimum Required Entities:

1. **Members** - Library card holders
    - Member ID, Name, Email, Phone, Address, Membership Type (Standard/Premium), Join Date, Expiry Date, Status (Active/Expired/Suspended)
2. **Books** - Physical book copies
    - Book ID, ISBN, Title, Edition, Publication Year, Total Copies, Available Copies, Shelf Location, Book Condition (New/Good/Fair/Damaged)
3. **Authors** - Book writers
    - Author ID, Name, Biography, Birth Year, Nationality
4. **Publishers** - Publishing companies
    - Publisher ID, Name, Country, Contact Email, Established Year
5. **Categories** - Book genres
    - Category ID, Category Name, Description

6. **Branches** - Library locations
    - Branch ID, Branch Name, Address, Phone, Manager Name, Opening Date
7. **Borrowing** - Loan transactions
    - Borrow ID, Member, Book, Branch, Borrow Date, Due Date, Return Date, Status (Borrowed/Returned/Overdue)
8. **Fines** - Overdue penalties
    - Fine ID, Borrow ID, Fine Amount, Paid Amount, Payment Date, Status (Pending/Paid/Waived)
9. **Reservations** - Book holds
    - Reservation ID, Member, Book, Reservation Date, Expiry Date, Status (Active/Fulfilled/Expired)

Required Relationships:

- Books ↔ Authors (Many-to-Many) - A book can have multiple authors, an author can write multiple books
- Books ↔ Categories (Many-to-Many) - A book can belong to multiple categories
- Books → Publishers (Many-to-One)
- Borrowing → Members, Books, Branches (Many-to-One each)
- Fines → Borrowing (One-to-One or One-to-Many)
- Reservations → Members, Books (Many-to-One each)

Business Rules to Implement:

- Members can borrow max 5 books at a time (Standard) or 10 books (Premium)
- Loan period: 14 days for Standard, 30 days for Premium
- Fine: $0.50 per day for overdue books
- Books can be reserved only if all copies are currently borrowed

---

Task 2: Normalization

Scenario: A junior developer created this denormalized table:

LibraryData Table:

| **member_id** | **member_name** | **book_id** | **book_title** | **author_names** | **categories** | **borrow_date** | **return_date** | **fine_amount** | **publisher_name** | **publisher_country** |

| 101 | John Doe | B001 | The Great Novel | Alice Smith, Bob Jones | Fiction, Drama | 2025-10-01 | 2025-10-20 | 5.00 | ABC Press | USA |

Your Task:

1. Identify normalization issues (1NF, 2NF, 3NF violations)
2. Normalize to 3NF showing each step
3. Explain the improvements and why each normal form matters
4. Show the final normalized tables

Task 3: Database Mapping

Convert your ERD into a complete relational schema:

- Define all tables with appropriate data types
- Show primary keys (PK) and foreign keys (FK)
- Identify junction tables for many-to-many relationships
- Document any constraints or indexes

Task 4: SQL DDL

Write SQL statements to:

1. **CREATE DATABASE** library_db
2. **CREATE all tables** with:
    - Appropriate data types (VARCHAR, INT, DATE, DECIMAL, ENUM, etc.)
    - Primary keys and auto-increment where needed
    - Foreign keys with ON DELETE and ON UPDATE rules
    - CHECK constraints (e.g., available_copies <= total_copies)
    - DEFAULT values (e.g., status = 'Active')
    - NOT NULL constraints where appropriate
3. **CREATE indexes** on:
    - Foreign keys
    - Search fields (book title, member name)
    - Date fields used in queries
4. **ALTER TABLE** statements to add at least 2 new columns:
    - Add last_login to Members
    - Add rating (1-5) to Books

Task 5: SQL DML

Populate the database with sample data:

INSERT minimum:

- 5 branches
- 20 members (mix of Standard/Premium, Active/Expired)
- 10 authors
- 5 publishers
- 8 categories
- 30 books (with varying availability)
- Book-Author relationships (some books with multiple authors)
- Book-Category relationships
- 25 borrowing records (include current, returned, and overdue)
- 8 fine records (some paid, some pending)
- 5 reservation records

UPDATE scenarios:

1. Update a member's membership from Standard to Premium
2. Mark books as returned and update available copies
3. Update fine status to 'Paid' and set payment date

DELETE scenarios:

1. Delete a reservation that expired
2. Attempt to delete a member who has active borrowings (should fail due to FK constraint)

Task 6: SQL DQL - Basic Queries

Write SELECT queries for:

1. List all books published after 2020
2. Find all members whose membership expires in the next 30 days
3. Show all overdue borrowings (return_date IS NULL AND due_date < CURDATE())
4. List books that have never been borrowed
5. Find members with pending fines greater than $10
6. Show all books in 'Fiction' category
7. Display books with less than 2 available copies

8. List all authors from 'USA' or 'UK'
9. Find books published by 'Penguin Random House'
10. Show borrowings from last month using BETWEEN

Task 7: SQL Joins

Write queries using different join types:

INNER JOIN:

1. List all borrowed books with member names and book titles
2. Show all books with their author names (handle multiple authors)
3. Display current borrowings with branch information

LEFT JOIN: 4. List ALL books and show if they're currently borrowed (include books not borrowed) 5. Show ALL members and their active borrowings (include members with no borrowings)

RIGHT JOIN / FULL OUTER JOIN (if supported): 6. Show all categories and count of books (include categories with no books)

Task 8: Aggregation Functions

Write queries using GROUP BY, HAVING, and aggregate functions:

1. **COUNT:**
    - Total number of borrowings per member
    - Number of books per category
    - Count of overdue books per branch
2. **SUM:**
    - Total fines collected per month
    - Total pending fines per member
    - Sum of available copies per publisher
3. **AVG:**
    - Average number of days books are borrowed
    - Average fine amount per member
    - Average books borrowed per branch
4. **MIN/MAX:**

- Find the oldest and newest publications in the library
- Member with maximum and minimum borrowings
- Highest fine amount ever charged

5. **Complex Aggregations with HAVING:**
   - Show categories with more than 5 books
   - Find members who borrowed more than 3 books in last month
   - List authors with more than 2 books in the library
   - Show branches with total pending fines > $100
6. **GROUP BY with Multiple Columns:**
   - Total borrowings per branch per month
   - Fine collection per member per year
7. **Subqueries with Aggregation:**
   - Find books borrowed more times than average
   - Members who paid fines above the average fine amount
